Universidad Da Vinci De Guatemala

Facultad de Ingeniería

Carrera: Ingeniería en Sistemas

“Primer Examen Parcial”

Nombre completo: Kerwin Sair Mendez Giron

Número de carnet: 202200969

Curso: Estructura de Datos

Guatemala, Marzo de 2026

Objetivo

El objetivo de esta práctica es comparar el rendimiento de algoritmos iterativos y recursivos mediante mediciones experimentales utilizando System.nanoTime(), analizando su complejidad temporal (Big-O) y contrastando los resultados experimentales con la teoría.

Marco Teórico

La notación Big-O se utiliza para describir cómo crece el tiempo de ejecución de un algoritmo conforme aumenta el tamaño de la entrada (n). No mide el tiempo exacto en milisegundos, sino el patrón de crecimiento.

Existen distintos tipos de crecimiento:

- O(1) = tiempo constante
- O(n) = crecimiento lineal
- O(n^2) = crecimiento cuadrático
- O(2^n) = crecimiento exponencial

En esta práctica se compararon versiones iterativas y recursivas de cuatro algoritmos para observar cómo influye la estructura del algoritmo en su rendimiento.

Metodología

Para realizar el análisis se implementaron cuatro algoritmos, cada uno en versión iterativa y recursiva:

- A1 = Factorial
- A2 = Fibonacci
- A3 = Búsqueda Lineal
- A4 = Ordenamiento Burbuja

El tiempo de ejecución se midió utilizando System.nanoTime() en Java.

Cada medición se ejecutó 5 veces para reducir el efecto del calentamiento de la JVM (JIT).

El promedio de esas cinco ejecuciones se calculó automáticamente en Excel.

En los algoritmos que trabajan con arreglos (Búsqueda y Burbuja), la generación del arreglo se realizó fuera del bloque de medición para evitar alterar los tiempos reales del algoritmo.

Los resultados se exportaron a un archivo CSV y posteriormente se graficaron en Excel usando gráficos de dispersión con líneas y marcadores.

Resultados

A1 – Factorial

Tanto la versión iterativa como la recursiva muestran crecimiento lineal.

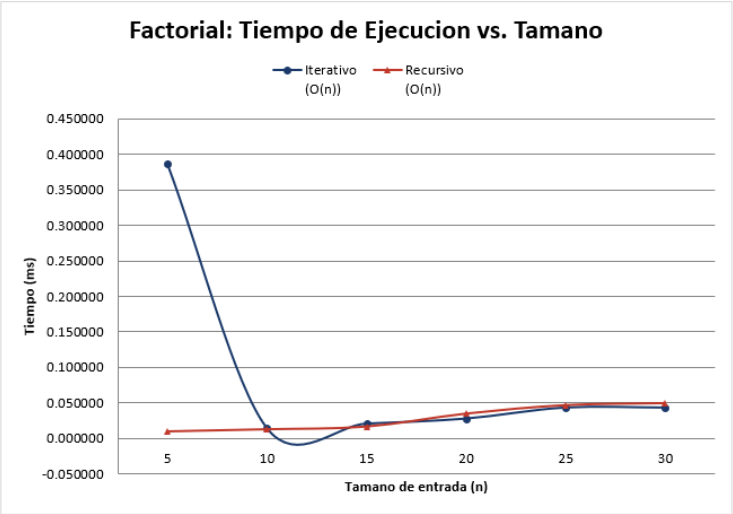
El tiempo aumenta conforme aumenta n, pero no se observan saltos grandes.

La versión recursiva tiene una ligera sobrecarga debido a las llamadas al stack, pero el comportamiento sigue siendo O(n).

HOJA — A1 - Factorial | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(n))
5	0.387560	0.009920
10	0.014540	0.012860
15	0.020720	0.016560
20	0.027840	0.034480
25	0.043420	0.046220
30	0.043380	0.048980

Observacion: Ambas versiones son O(n): el tiempo crece linealmente con n.



A2 – Fibonacci

Aquí se observa la mayor diferencia.

La versión iterativa mantiene un comportamiento estable y lineal.

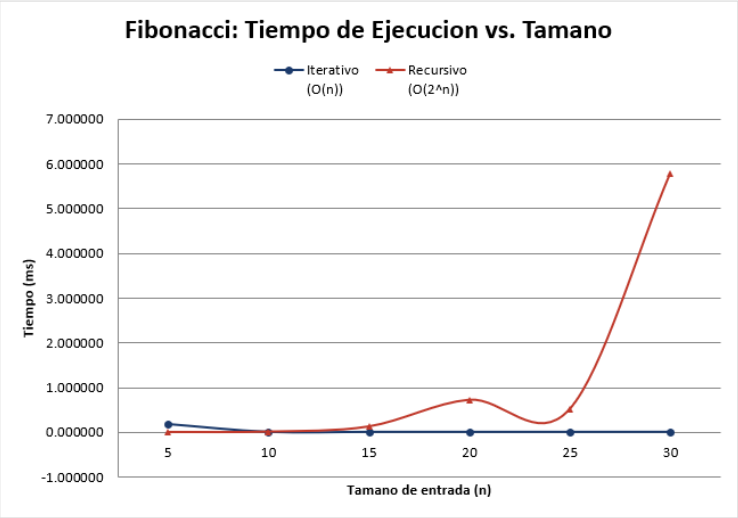
En cambio, la versión recursiva comienza a crecer rápidamente a partir de n=20 y en n=30 el tiempo aumenta considerablemente.

Esto confirma que la versión recursiva sin memoización tiene complejidad exponencial.

HOJA — A2 - Fibonacci | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(2^n))
5	0.174680	0.002920
10	0.000700	0.011980
15	0.000920	0.126780
20	0.000720	0.725120
25	0.000840	0.514520
30	0.001100	5.792080

Observacion: Recursivo crece exponencialmente: duplicar n casi cuadriplica el tiempo.



A3 – Búsqueda Lineal

Ambas versiones presentan crecimiento lineal.

Al aumentar el tamaño del arreglo, el tiempo también aumenta proporcionalmente.

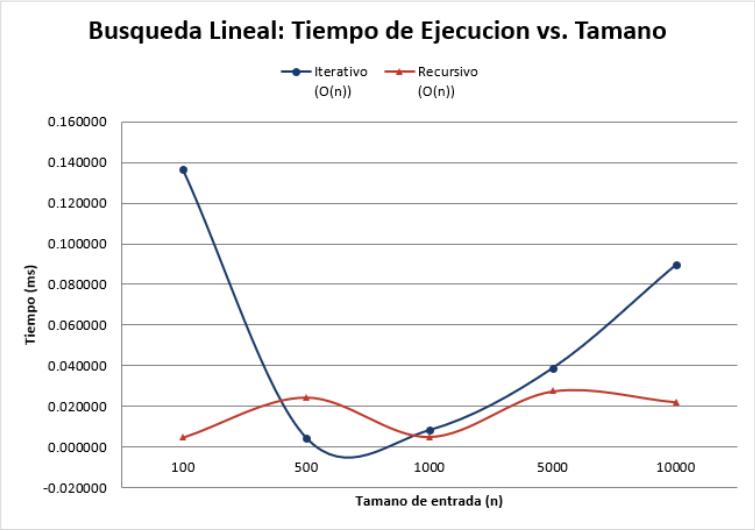
Las pequeñas variaciones se deben al entorno de ejecución, pero el patrón general se mantiene como O(n).

HOJA — A3 - Busqueda Lineal | Iterativo vs. Recursivo

Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(n))
100	0.136740	0.004740
500	0.004620	0.024560
1000	0.008380	0.005120
5000	0.038960	0.027580
10000	0.089880	0.022200

Observacion: Ambas O(n): al multiplicar n x10, el tiempo tambien x10 aprox.



A4 – Ordenamiento Burbuja

En este caso el crecimiento es mucho más pronunciado.

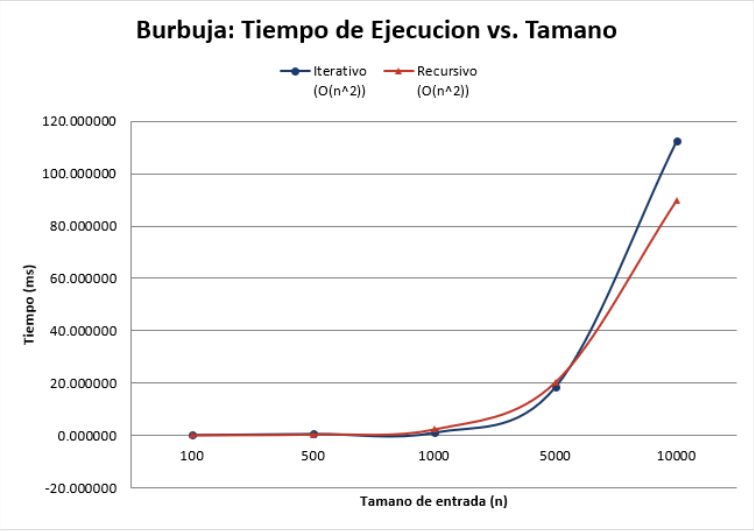
Al aumentar el tamaño del arreglo de 1000 a 10000, el tiempo aumenta significativamente.

Esto confirma que el algoritmo tiene complejidad O(n^2), tanto en su versión iterativa como recursiva.

HOJA — A4 - Burbuja | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n^2)	Big-O Recursivo:

n	Iterativo (O(n^2))	Recursivo (O(n^2))
100	0.321300	0.163260
500	0.745100	0.433180
1000	1.246600	2.348060
5000	18.724720	20.431280
10000	112.648780	90.031520

Observacion: Ambas O(n^2): al x10 n, el tiempo aumenta aprox x100.



Los resultados muestran claramente diferentes patrones de crecimiento según el algoritmo evaluado.

En general:

- Los algoritmos lineales presentan crecimiento proporcional a n.
- Burbuja presenta crecimiento cuadrático.
- Fibonacci recursivo presenta crecimiento exponencial notable.

Conclusión General

Los resultados experimentales obtenidos en esta práctica coinciden con las complejidades teóricas esperadas.

Se comprobó que:

- Los algoritmos lineales (Factorial y Búsqueda Lineal) crecen proporcionalmente al tamaño de entrada.
- El algoritmo Burbuja presenta crecimiento cuadrático.
- Fibonacci recursivo demuestra claramente crecimiento exponencial, lo que lo hace poco eficiente para valores grandes de n.

Una conclusión importante es que la recursión no siempre significa menor eficiencia.

En el caso de Fibonacci, la versión recursiva es más elegante, pero su rendimiento es muy inferior al iterativo cuando n aumenta.

En general, esta práctica permitió comprobar de forma experimental cómo se comportan los algoritmos en la realidad y cómo la notación Big-O se refleja en los tiempos medidos.

Referencias

- GeeksforGeeks. (2023). Analysis of Algorithms | Big-O Analysis. Recuperado de: https://www.geeksforgeeks.org/analysis-of-algorithms-big-o-analysis/
- GeeksforGeeks. (2023). Time Complexity of Recursive Fibonacci. Recuperado de: https://www.geeksforgeeks.org/time-complexity-recursive-fibonacci-program/
- GeeksforGeeks. (2023). Bubble Sort. Recuperado de: https://www.geeksforgeeks.org/bubble-sort/
- GeeksforGeeks. (2023). Linear Search. Recuperado de: https://www.geeksforgeeks.org/linear-search/
- Oracle. (2023). System.nanoTime() Method. Recuperado de: https://docs.oracle.com/javase/8/docs/api/java/lang/System.html#nanoTime--